GUÍA DE LABORATORIO

Dashboard con React + Supabase

Paso a paso para crear una app con registro, inicio de sesión, perfil e imágenes, y un dashboard de administración protegido con RLS. Todo gratis y desde GitHub Codespaces.

React + Vite | Supabase Auth | PostgreSQL | Row Level Security

Material para estudiantes · Desarrollo web y móvil

Versión 2 · noviembre de 2026

Lo esencial en una página

Si solo tienes cinco minutos antes de la clase, lee esto.

QUÉ VAS A CONSTRUIR

Una app en React donde las personas se registran, inician sesión y guardan sus imágenes, y un **dashboard de administración** con indicadores, gráfica y tabla editable. Los datos y los usuarios viven en Supabase, gratis.

Antes de empezar necesitas

- Una cuenta de **GitHub** (con ella entras a Supabase).
- Un proyecto **React + Vite** abierto en GitHub Codespaces.
- Saber qué es un componente, `useState` y `useEffect`.
- Nada de CSS propio: todo el diseño es **Bootstrap 5**.
- El proyecto terminado está en **github.com/sdtibata/React-supabase**. Úsalo para comparar si algo no te funciona.
- Unas **3 horas**, idealmente en dos sesiones.

Las 4 reglas que nunca se rompen

1. **Las claves van en `.env`**, y `.env` nunca se sube a GitHub.
2. **En el frontend solo va la clave publicable.** La secreta nunca.
3. **RLS siempre activo.** Sin políticas, tus datos quedan expuestos.
4. **El rol se cambia solo desde Supabase**, nunca desde la app.

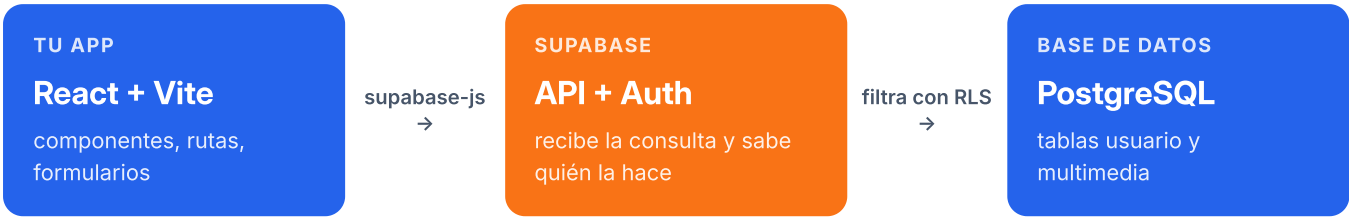
Contenido

1 Cómo encajan las piezas
2 Crear el proyecto en Supabase
3 Conectar React con Supabase
4 Crear las tablas
5 Perfil automático al registrarse
6 Seguridad con RLS
7 Usuarios y datos de prueba
8 La app: login, registro y perfil
9 El dashboard
10 Convertir en app Android
11 Errores comunes
A Anexo: pedirle ayuda a Claude

Todo el código de esta guía se probó en un proyecto real de Supabase. Las capturas son de noviembre de 2026; si la interfaz cambia un poco, los nombres de los menús suelen ser los mismos.

1 Cómo encajan las piezas

Supabase es un **BaaS** (Backend as a Service): te da base de datos, usuarios y API sin que tengas que programar ni pagar un servidor.



Pieza	Qué hace	Dónde la ves en Supabase
Database	Guarda tus tablas. Es PostgreSQL, el mismo SQL que ves en la clase de bases de datos.	Table Editor, SQL Editor
Auth	Crea cuentas, guarda contraseñas cifradas y abre sesiones.	Authentication
API	Convierte cada tabla en una API. Desde React escribes `supabase.from('usuario')` y no un `fetch` a mano.	Se crea sola
RLS	Row Level Security: reglas que deciden qué filas puede ver o cambiar cada usuario.	Database → Policies

Las dos claves de tu proyecto

Publishable (pública)

Empieza por `sb_publishable_`. Va en tu app React. Cualquiera que abra la página puede verla, y está bien: lo que protege tus datos es RLS, no esta clave.

Secret (secreta)

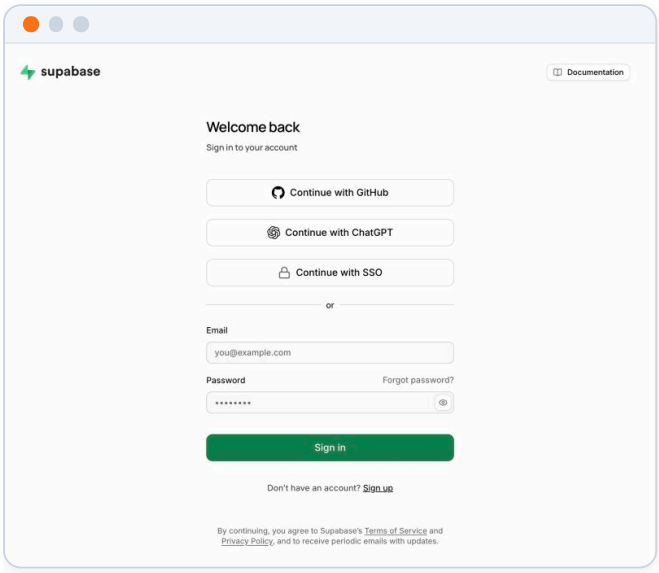
Empieza por `sb_secret_`. Se salta todas las reglas de RLS. **Nunca** va en React, ni en GitHub, ni en un mensaje de WhatsApp.

En guías viejas verás `anon` y `service_role`. Son las mismas dos claves con el nombre anterior: `anon` = publishable, `service_role` = secret.

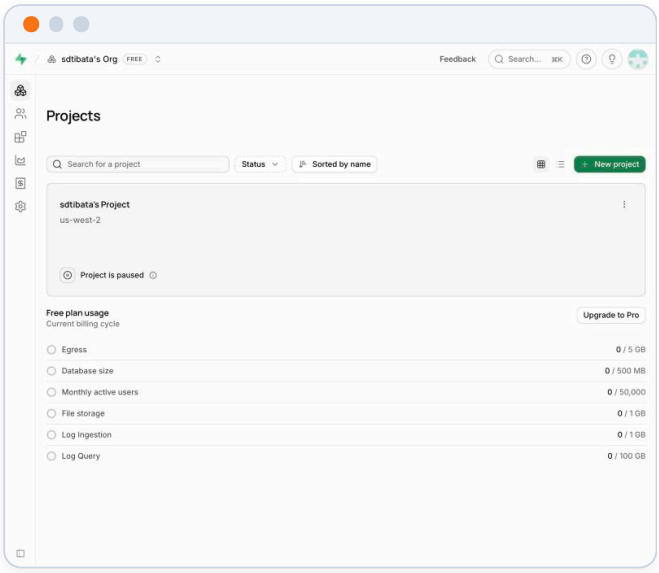
2 Crear el proyecto en Supabase

Un proyecto de Supabase es tu base de datos en la nube. El plan gratuito permite dos proyectos activos.

1. Entra a **supabase.com/dashboard** y elige **Continue with GitHub**.
2. Abre tu organización y haz clic en **New project**.

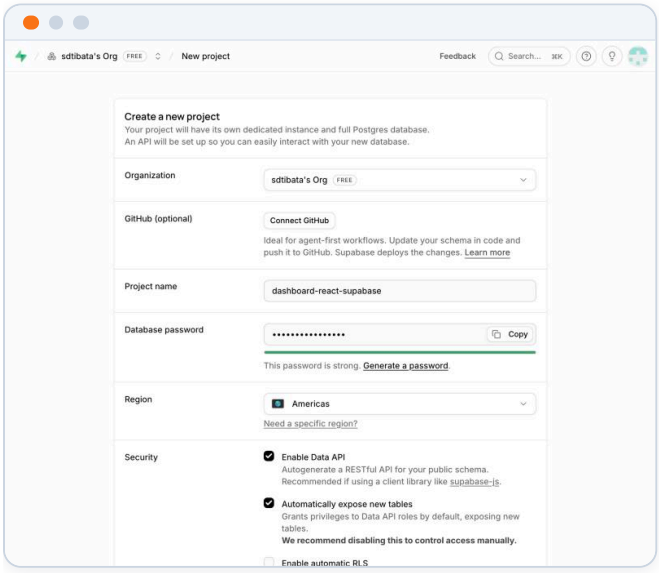


Inicio de sesión. Usa tu cuenta de GitHub.

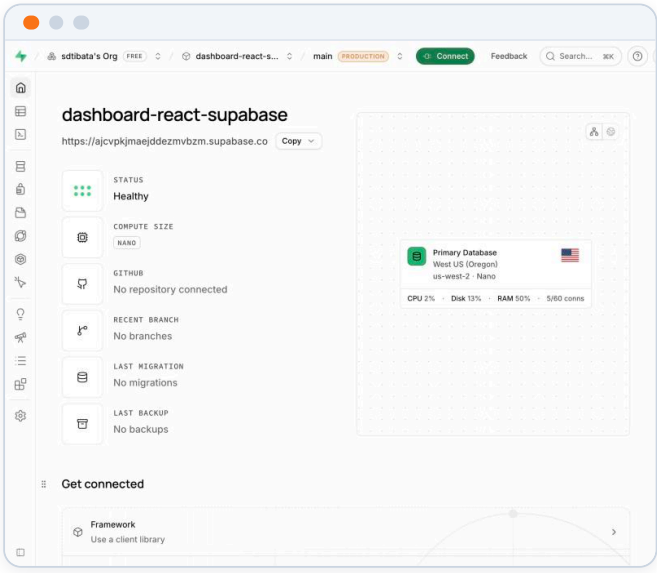


Proyectos. El botón verde *New project* está arriba a la derecha.

3. Escribe el nombre del proyecto (por ejemplo `dashboard-react-supabase`) y haz clic en **Generate a password**. Guarda esa contraseña en un lugar seguro: es la de la base de datos, no la de tu cuenta.
4. Deja la región **Americas** y las opciones de seguridad como vienen. Haz clic en **Create new project** y espera uno o dos minutos.



Nombre y contraseña. La contraseña generada aparece con la barra verde.

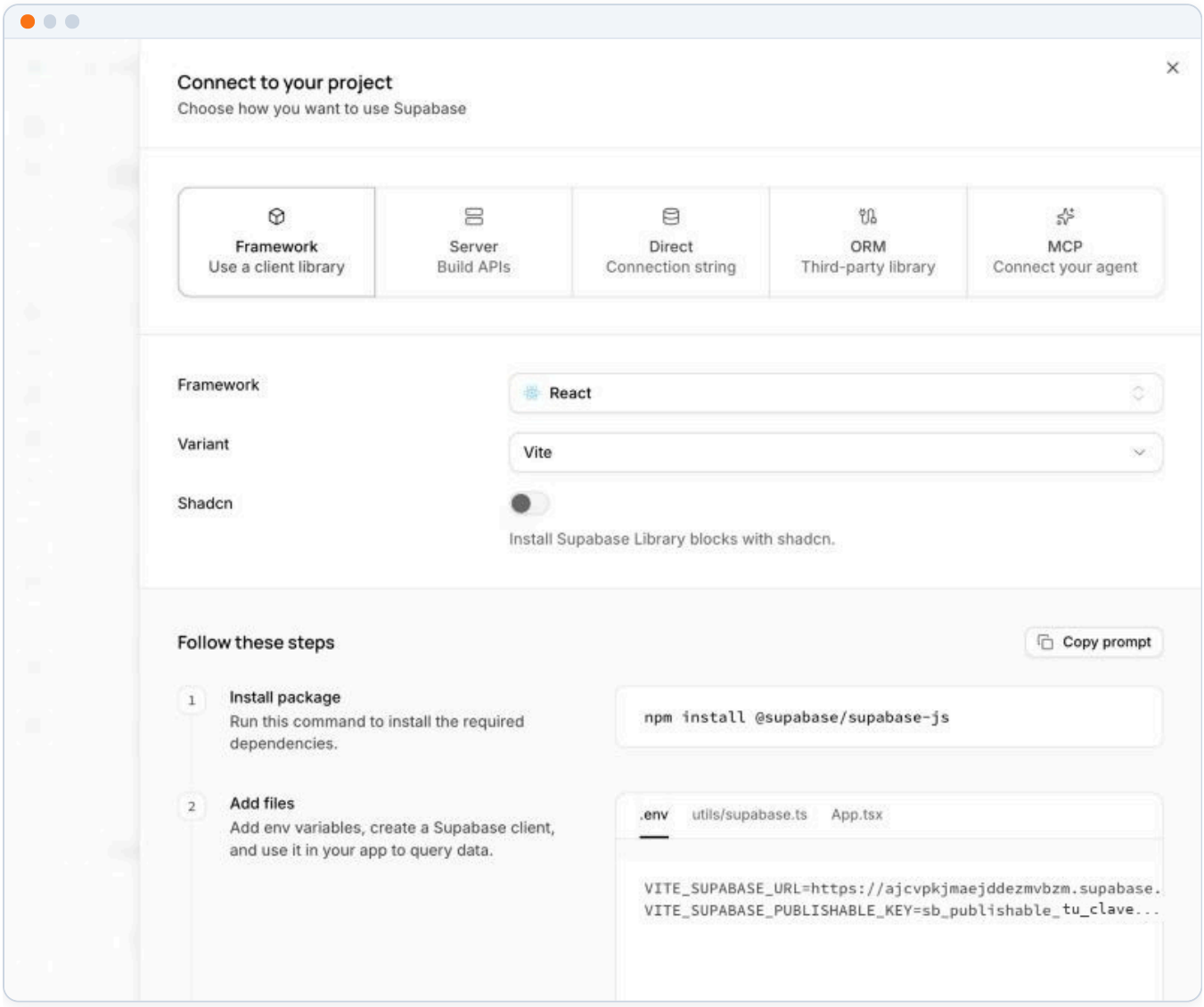


Proyecto listo. Debajo del nombre está la URL de tu proyecto.

3 Conectar React con Supabase

Necesitas dos datos: la URL del proyecto y la clave publicable. Supabase te los da listos para Vite.

1 En la parte de arriba del proyecto haz clic en **Connect**. En *Framework* elige **React** y en *Variant* elige **Vite**.



Connect → React → Vite. En la pestaña `.env` están tus dos variables. Cópialas tal cual.

2 En la terminal del Codespace instala la librería de Supabase, el enrutador y Bootstrap:

```
Terminal
```

```
npm install @supabase/supabase-js react-router-dom bootstrap
```

3. Crea el archivo `.env` en la **raíz del proyecto** (al lado de `package.json`, no dentro de `src`) y pega tus valores:

```
.env

VITE_SUPABASE_URL=https://tu-proyecto.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_tu_clave
```

4. Agrega `.env` al `.gitignore` y crea un `.env.example` vacío. Ese sí se sube, para que tus compañeros sepan qué variables llenar.

```
.gitignore (agregar al final)

# variables de entorno: nunca a GitHub
.env
.env.local
```

```
.env.example

VITE_SUPABASE_URL=
VITE_SUPABASE_PUBLISHABLE_KEY=
```

5. Crea `src/supabase.js`. Este archivo se importa en todos los componentes que hablan con Supabase.

```
src/supabase.js

// src/supabase.js
import { createClient } from '@supabase/supabase-js'

export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY
)
```

6. Reemplaza `src/main.jsx` para cargar Bootstrap. Borra `index.css` y `App.css` (y sus `import`): no los vamos a usar.

```
src/main.jsx

// src/main.jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import 'bootstrap/dist/css/bootstrap.min.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
)
```

Ojo: Vite solo lee el .env al arrancar

Si creaste o cambiaste el `.env` con `npm run dev` corriendo, detenlo (Ctrl + C) y vuelve a correrlo. Si no, `import.meta.env.VITE_SUPABASE_URL` llega como `undefined` y verás el error *supabaseUrl is required*. Las variables tienen que empezar por `VITE_` para que Vite las deje pasar al navegador.

4 Crear las tablas

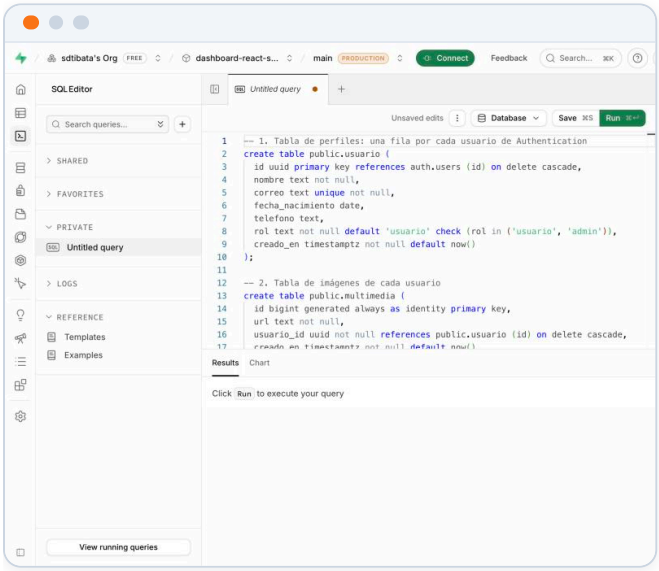
Dos tablas: **usuario** (el perfil de cada persona) y **multimedia** (sus imágenes). Se crean con SQL en el SQL Editor.

1 En el menú de la izquierda abre **SQL Editor** (el ícono `>_`), pega este código y haz clic en **Run**.

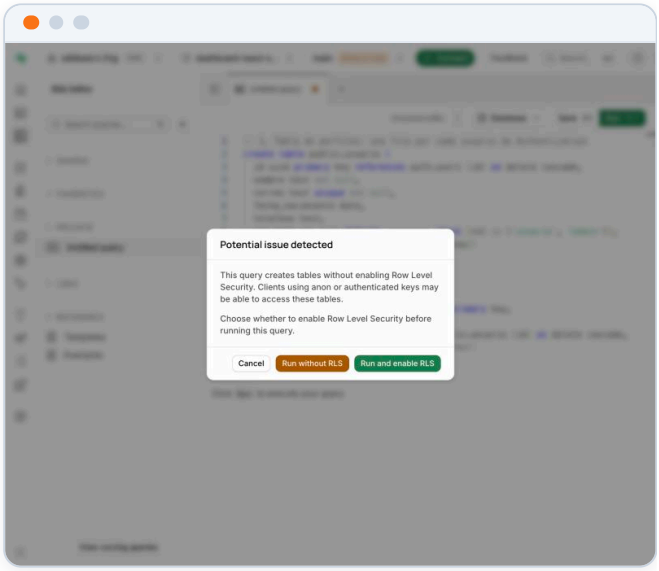
SQL Editor · tablas

```sql
-- 1. Tabla de perfiles: una fila por cada usuario de Authentication
create table public.usuario (
  id uuid primary key references auth.users (id) on delete cascade,
  nombre text not null,
  correo text unique not null,
  fecha_nacimiento date,
  telefono text,
  rol text not null default 'usuario' check (rol in ('usuario', 'admin')),
  creado_en timestamptz not null default now()
);

-- 2. Tabla de imágenes de cada usuario
create table public.multimedia (
  id bigint generated always as identity primary key,
  url text not null,
  usuario_id uuid not null references public.usuario (id) on delete cascade,
  creado_en timestamptz not null default now()
);
```

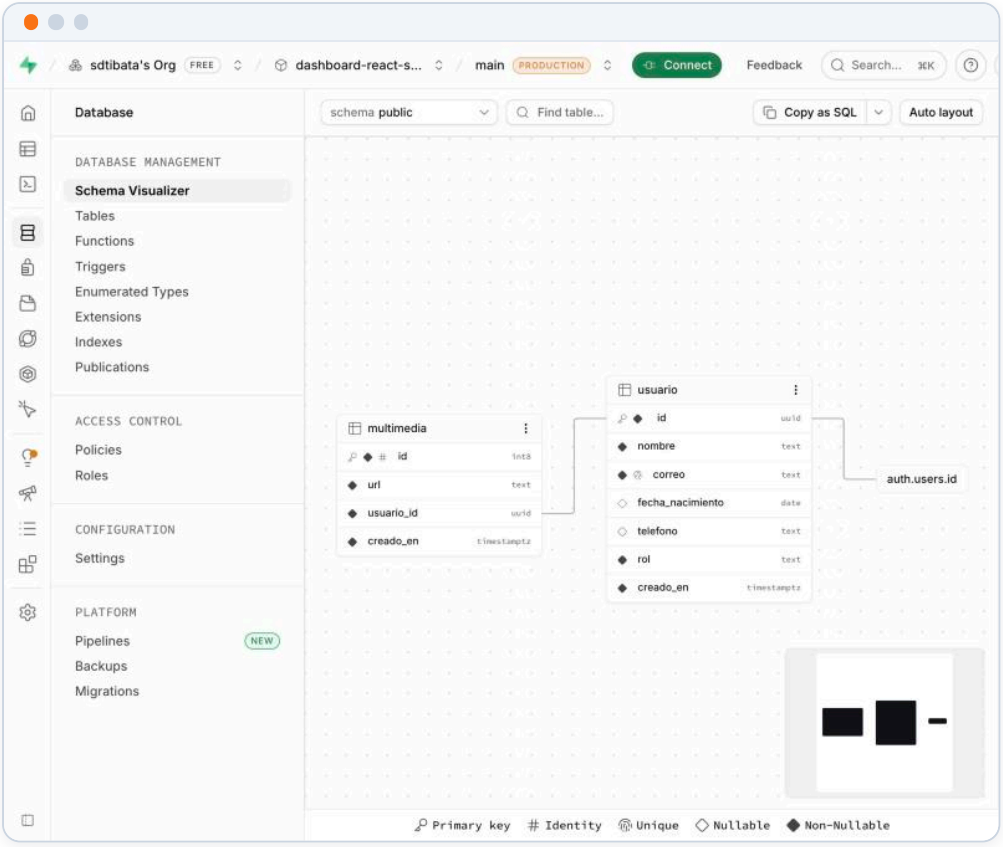


SQL Editor. El botón *Run* está arriba a la derecha.



Aviso de RLS. Elige **Run and enable RLS**, el botón verde.

2 Debe aparecer **Success. No rows returned**. Para ver las tablas y cómo se conectan, ve a **Database → Schema Visualizer**.

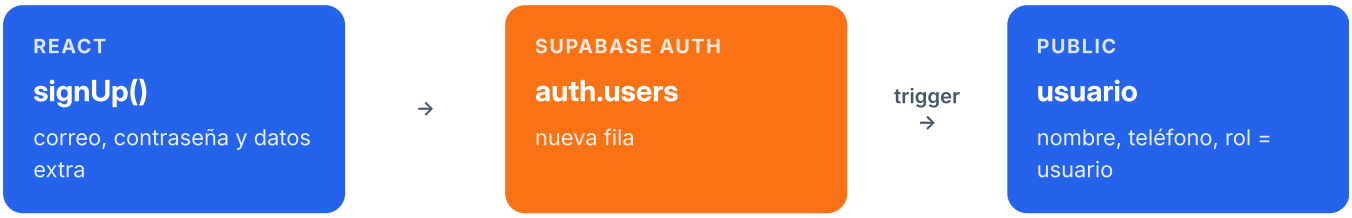


Schema Visualizer. `multimedia.usuario_id` apunta a `usuario.id`, y `usuario.id` apunta a `auth.users.id`: la tabla interna de Supabase donde viven las cuentas.

Columna	Por qué está así
`usuario.id`	Es el **mismo id** de la cuenta en Authentication. Así cada perfil queda amarrado a su cuenta, y si se borra la cuenta se borra el perfil (`on delete cascade`).
`rol`	Solo acepta `'usuario'` o `'admin'` (`check`). Por defecto todos son `usuario`.
`multimedia.id`	`generated always as identity` es la forma moderna de `SERIAL`: un número que sube solo.
`creado_en`	Fecha automática. El dashboard la usa para contar los usuarios nuevos de la semana.

5 Perfil automático al registrarse

Cuando alguien se registra, Supabase guarda la cuenta en `auth.users`. Pero nuestros datos (nombre, teléfono, rol) están en `usuario`. Un **trigger** copia la fila de una tabla a la otra sin que React haga nada.



1 En una consulta nueva del SQL Editor (botón **+**) pega y ejecuta:

SQL Editor · trigger

```sql
-- Cuando alguien se registra, Supabase crea su fila en auth.users.
-- Este trigger copia sus datos a public.usuario automáticamente.
create function public.crear_perfil()
returns trigger
language plpgsql
security definer set search_path = ''
as $$
begin
  insert into public.usuario (id, nombre, correo, fecha_nacimiento, telefono)
  values (
    new.id,
    coalesce(new.raw_user_meta_data ->> 'nombre', split_part(new.email, '@', 1)),
    new.email,
    nullif(new.raw_user_meta_data ->> 'fecha_nacimiento', '')::date,
    new.raw_user_meta_data ->> 'telefono'
  );
  return new;
end;
$$;

create trigger al_registrarse
  after insert on auth.users
  for each row execute function public.crear_perfil();
```

¿Por qué no insertar el perfil desde React, como en la versión anterior?

Si el proyecto pide confirmar el correo, justo después del `signUp` todavía no hay sesión. Con RLS activo, el `insert` desde React falla y queda una cuenta sin perfil. El trigger corre dentro de la base de datos, así que nunca falla por eso.

6 Seguridad con RLS

Con RLS activo y sin políticas, nadie puede leer nada. Las políticas abren solo lo necesario. Este es el contrato de nuestra app:

Acción	Usuario normal	Admin
Ver perfiles	Solo el suyo	Todos
Editar perfiles	Solo el suyo, sin tocar el rol	Todos
Ver imágenes	Solo las suyas	Todas
Agregar imágenes	Solo a su nombre	Solo a su nombre
Borrar imágenes	Solo las suyas	Todas

1 En una consulta nueva pega y ejecuta todo el bloque:

SQL Editor · políticas RLS

```sql
-- ¿El usuario que hace la consulta es admin?
create function public.es_admin()
returns boolean
language sql
stable
security definer set search_path = ''
as $$
  select exists (
    select 1 from public.usuario
    where id = auth.uid() and rol = 'admin'
  );
$$;

-- Activar RLS: sin políticas, nadie puede leer ni escribir
alter table public.usuario enable row level security;
alter table public.multimedia enable row level security;

-- USUARIO: cada quien ve y edita lo suyo; el admin ve y edita todo
create policy "ver perfil propio o admin" on public.usuario
  for select to authenticated
  using (id = auth.uid() or public.es_admin());

create policy "editar perfil propio o admin" on public.usuario
  for update to authenticated
  using (id = auth.uid() or public.es_admin());

-- Nadie puede cambiarse el rol desde la app: solo un admin o el panel de Supabase
create function public.proteger_rol()
returns trigger
language plpgsql
security definer set search_path = ''
as $$
begin
  -- auth.uid() es null cuando editas desde el panel de Supabase
  if new.rol <> old.rol and auth.uid() is not null and not public.es_admin() then
    raise exception 'Solo un admin puede cambiar el rol';
```

```
  end if;
  return new;
end;
$$;

create trigger antes_de_editar_usuario
  before update on public.usuario
  for each row execute function public.proteger_rol();

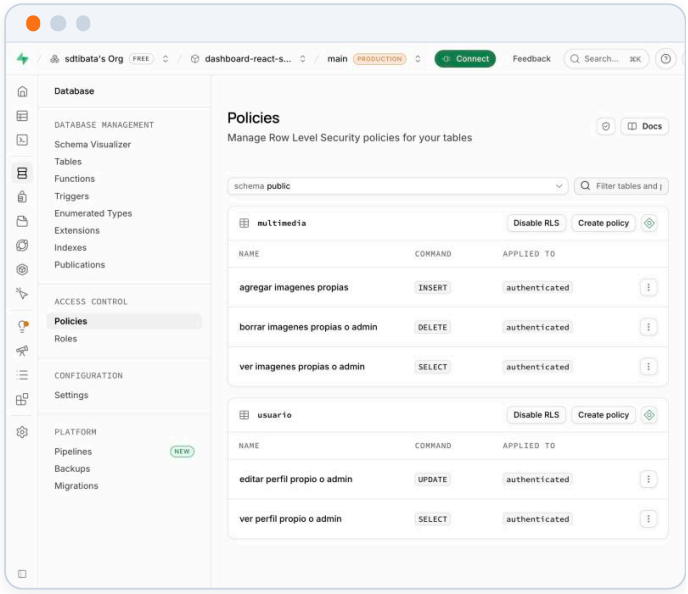
-- MULTIMEDIA: cada quien ve, agrega y borra sus imágenes; el admin todo
create policy "ver imagenes propias o admin" on public.multimedia
  for select to authenticated
  using (usuario_id = auth.uid() or public.es_admin());

create policy "agregar imagenes propias" on public.multimedia
  for insert to authenticated
  with check (usuario_id = auth.uid());

create policy "borrar imagenes propias o admin" on public.multimedia
  for delete to authenticated
  using (usuario_id = auth.uid() or public.es_admin());
```

Qué hace cada parte

- `es_admin()` responde si quien consulta es admin. Va en una función con `security definer` porque, si la política de `usuario` consultara la misma tabla `usuario`, entraría en un ciclo infinito.
- `auth.uid()` es el id del usuario que tiene la sesión abierta. Supabase lo saca del token que manda supabase-js.
- `using` decide qué filas puede ver, editar o borrar. `with check` decide qué filas puede crear.
- `proteger_rol` es un trigger que frena a cualquiera que intente volverse admin desde la app.

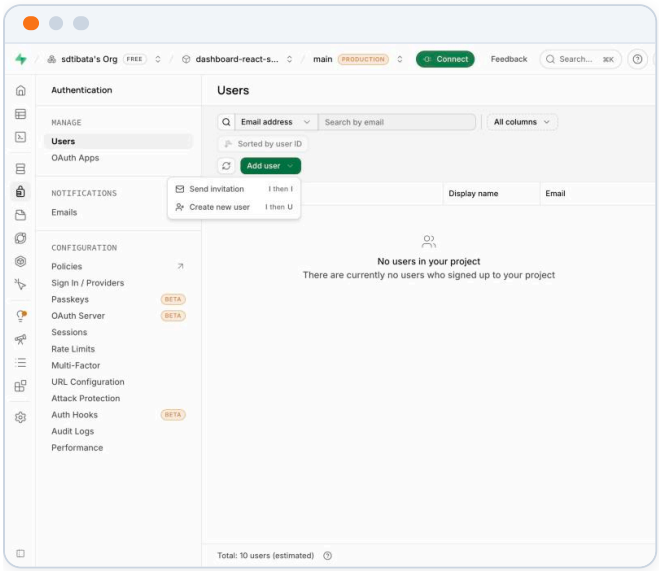


Database → Policies. Deben aparecer 3 políticas en `multimedia` y 2 en `usuario`.

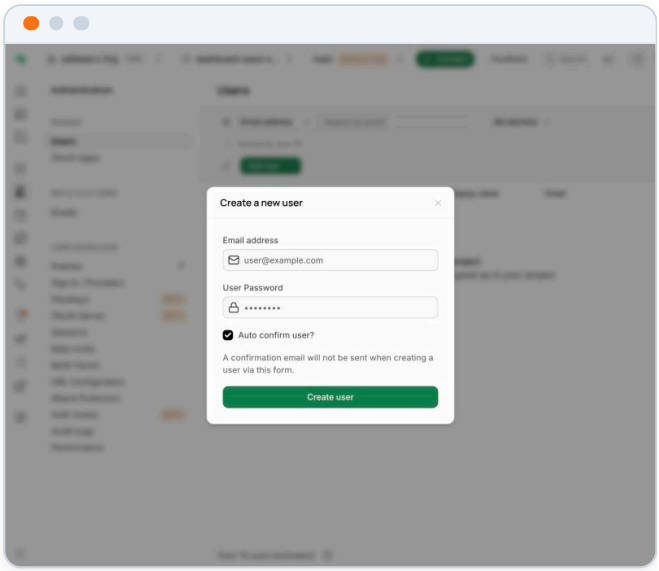
7 Usuarios y datos de prueba

Para que el dashboard tenga algo que mostrar, crea tres usuarios: uno será admin y dos serán usuarios normales.

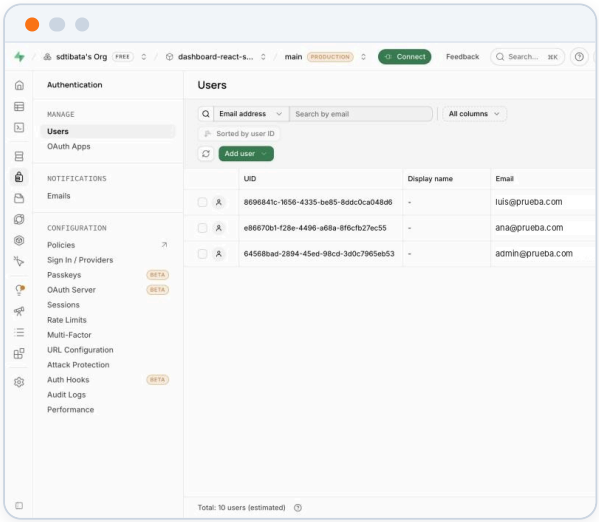
1. Ve a **Authentication → Users → Add user → Create new user**. Escribe un correo de prueba y una contraseña, y deja marcado **Auto confirm user**. Repite para los tres: `admin@prueba.com`, `ana@prueba.com` y `luis@prueba.com`.



Add user. Elige *Create new user*.



Auto confirm user evita tener que abrir el correo de confirmación.



Los tres usuarios creados. El trigger del paso 5 ya les creó su fila en `usuario`.

Para clase: el correo de confirmación

En **Authentication → Sign In / Providers** está activo **Confirm email**: quien se registre desde la app recibe un correo y no puede entrar hasta confirmarlo. El plan gratis envía pocos correos por hora, así que para practicar en clase puedes apagarlo y hacer clic en *Save changes*. Vuelve a encenderlo antes de entregar.

2 Vuelve al SQL Editor para nombrar al admin, completar datos y agregar imágenes de ejemplo:

SQL Editor · datos de prueba

```sql
-- Volver admin a un usuario (solo se puede desde Supabase)
update public.usuario set rol = 'admin', nombre = 'Admin Prueba'
where correo = 'admin@prueba.com';

-- Completar datos de los otros dos
update public.usuario set nombre = 'Ana Rojas', telefono = '3109876543',
  fecha_nacimiento = '2003-04-02'
where correo = 'ana@prueba.com';

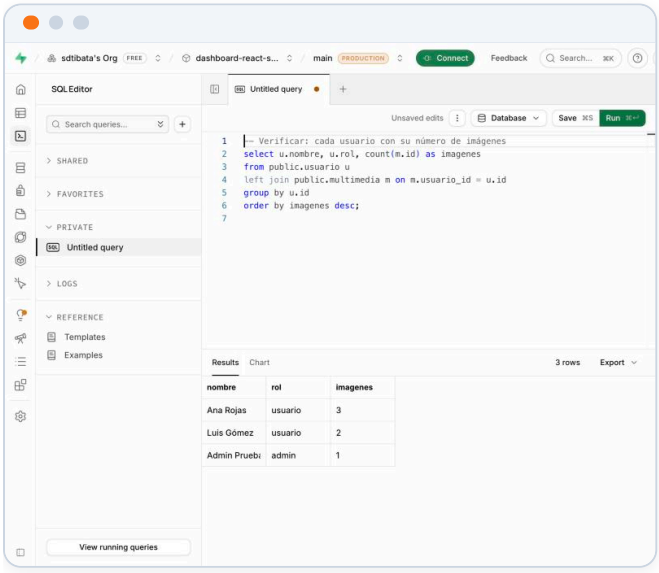
update public.usuario set nombre = 'Luis Gómez', telefono = '3205550101',
  fecha_nacimiento = '2002-11-20'
where correo = 'luis@prueba.com';

-- Imágenes de ejemplo
insert into public.multimedia (url, usuario_id)
select 'https://picsum.photos/id/' || n || '/400/400', u.id
from public.usuario u
join (values ('admin@prueba.com', 10), ('ana@prueba.com', 20),
             ('ana@prueba.com', 28), ('ana@prueba.com', 29),
             ('luis@prueba.com', 37), ('luis@prueba.com', 42)) as d(correo, n)
  on d.correo = u.correo;
```

3 Comprueba que todo quedó bien con un `join`. Ana debe tener 3 imágenes, Luis 2 y el admin 1.

SQL Editor · verificar

```sql
-- Verificar: cada usuario con su número de imágenes
select u.nombre, u.rol, count(m.id) as imagenes
from public.usuario u
left join public.multimedia m on m.usuario_id = u.id
group by u.id
order by imagenes desc;
```



Resultado del join. Un usuario por fila con su número de imágenes.

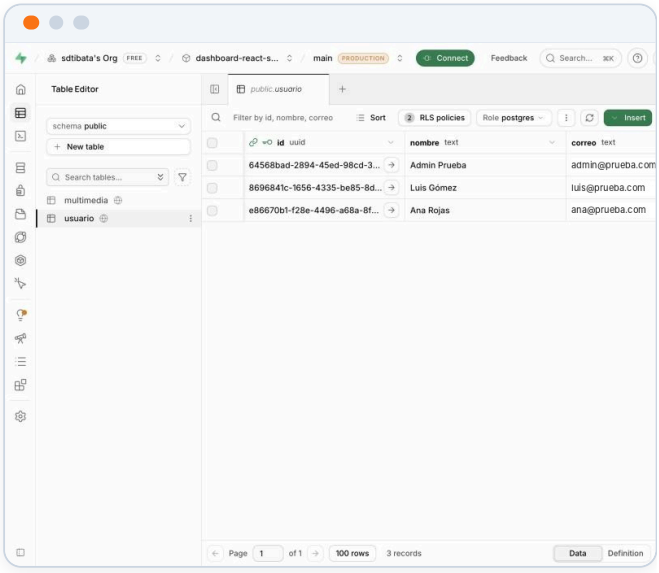


Table Editor → usuario. Doble clic sobre una celda para editarla a mano, por ejemplo el rol.

8 La app: login, registro y perfil

Cuatro pantallas y un archivo de rutas. Si ya tienes un proyecto (por ejemplo la PokeAPI), agrega estos componentes y ajusta los `import` a tus carpetas. Todos los archivos están completos en **github.com/sdtibata/React-supabase**: ábrelos allá y usa **Copy raw file** para copiarlos sin errores.

Estructura de carpetas

src/

```
src/
├── main.jsx          (importa Bootstrap)
├── App.jsx
├── supabase.js
└── componentes/
    ├── login/index.jsx
    ├── registro/index.jsx
    ├── perfil/index.jsx
    └── dashboard/index.jsx
```

Rutas de la app

`/login`	Pública. Si ya hay sesión, manda al perfil.
`/registro`	Pública.
`/perfil`	Privada. Cualquier usuario con sesión.
`/dashboard`	Privada. El componente revisa que sea admin.

App.jsx: rutas y sesión

`getSession()` revisa si ya había una sesión abierta (por ejemplo, al recargar la página) y `onAuthStateChange` avisa cada vez que alguien entra o sale. La función `privada()` manda al login a quien no tenga sesión.

src/App.jsx

```jsx
// src/App.jsx
import { useEffect, useState } from 'react'
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
import { supabase } from './supabase'
import Login from './componentes/login'
import Registro from './componentes/registro'
import Perfil from './componentes/perfil'
import Dashboard from './componentes/dashboard'

function App() {
  const [usuario, setUsuario] = useState(null)
  const [cargando, setCargando] = useState(true)

  useEffect(() => {
    // 1. ¿Ya había una sesión abierta?
    supabase.auth.getSession().then(({ data: { session } }) => {
      setUsuario(session?.user ?? null)
      setCargando(false)
    })

    // 2. Escuchar cuando alguien entra o sale
    const { data: { subscription } } = supabase.auth.onAuthStateChange(
      (_evento, session) => setUsuario(session?.user ?? null)
    )
    return () => subscription.unsubscribe()
  }, [])
```

```jsx
  if (cargando) return <p>Cargando...</p>

  // Si no hay sesión, cualquier ruta privada manda al login
  const privada = (componente) => (usuario ? componente : <Navigate to="/login" />)

  return (
    <BrowserRouter>
      <Routes>
        <Route path="/login" element={usuario ? <Navigate to="/perfil" /> : <Login />} />
        <Route path="/registro" element={<Registro />} />
        <Route path="/perfil" element={privada(<Perfil />)} />
        <Route path="/dashboard" element={privada(<Dashboard />)} />
        <Route path="*" element={<Navigate to="/perfil" />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

Todas las pantallas usan solo clases de Bootstrap (`container`, `card`, `form-control`, `btn`...). No hay ni una línea de CSS propio.

Login

`signInWithPassword` recibe correo y contraseña. Si funciona, Supabase guarda la sesión en el navegador y `onAuthStateChange` en `App.jsx` se entera solo.



Pantalla de login.

src/componentes/login/index.jsx

```jsx
// src/componentes/login/index.jsx
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { supabase } from '../../supabase'

function Login() {
  const [correo, setCorreo] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState(null)
  const navigate = useNavigate()

  const handleLogin = async (e) => {
    e.preventDefault()
```

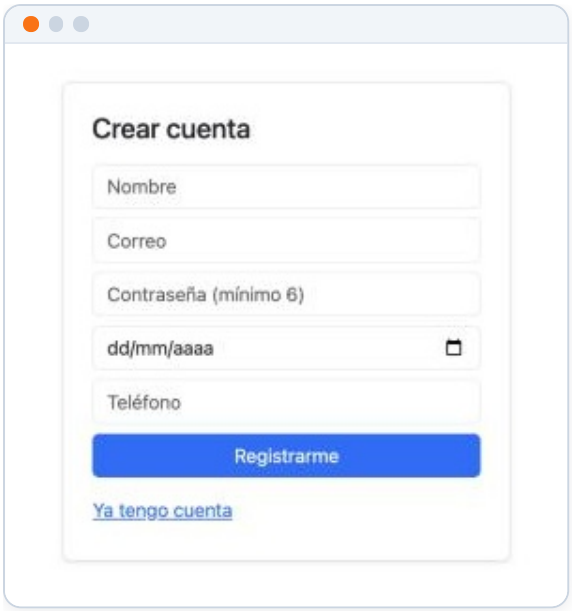
```
    const { error } = await supabase.auth.signInWithPassword({
      email: correo,
      password,
    })
    if (error) setError('Correo o contraseña no válidos')
    else navigate('/perfil')
  }

  return (
    <div className="container py-5">
      <div className="row justify-content-center">
        <div className="col-md-5">
          <div className="card shadow-sm">
            <div className="card-body p-4">
              <h2 className="h4 mb-3">Iniciar sesión</h2>
              <form onSubmit={handleLogin} className="d-grid gap-2">
                <input className="form-control" type="email"
                  placeholder="Correo" value={correo}
                  onChange={(e) => setCorreo(e.target.value)} required />
                <input className="form-control" type="password"
                  placeholder="Contraseña" value={password}
                  onChange={(e) => setPassword(e.target.value)} required />
                <button className="btn btn-primary">Entrar</button>
              </form>
              {error && <div className="alert alert-danger mt-3">{error}</div>}
              <button className="btn btn-link px-0 mt-2"
                onClick={() => navigate('/registro')}>
                ¿No tienes cuenta? Regístrate
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>
  )
}

export default Login
```

Registro

El nombre, la fecha y el teléfono viajan en `options.data`. Supabase los guarda en `raw_user_meta_data`, de donde los lee el trigger del paso 5. Por eso este componente no hace ningún `insert`.



Pantalla de registro.

src/componentes/registro/index.jsx

```jsx
// src/componentes/registro/index.jsx
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { supabase } from '../../supabase'

function Registro() {
  const [form, setForm] = useState({
    nombre: '', correo: '', password: '', fecha_nacimiento: '', telefono: '',
  })
  const [mensaje, setMensaje] = useState(null)
  const navigate = useNavigate()

  const handleChange = (e) =>
    setForm({ ...form, [e.target.name]: e.target.value })

  const handleRegistro = async (e) => {
    e.preventDefault()
    const { error } = await supabase.auth.signUp({
      email: form.correo,
      password: form.password,
      // Estos datos los lee el trigger y crea la fila en "usuario"
      options: {
        data: {
          nombre: form.nombre,
          fecha_nacimiento: form.fecha_nacimiento,
          telefono: form.telefono,
        },
      },
    })
    if (error) setMensaje(error.message)
    else setMensaje('Cuenta creada. Confirma tu correo y luego inicia sesión.')
  }

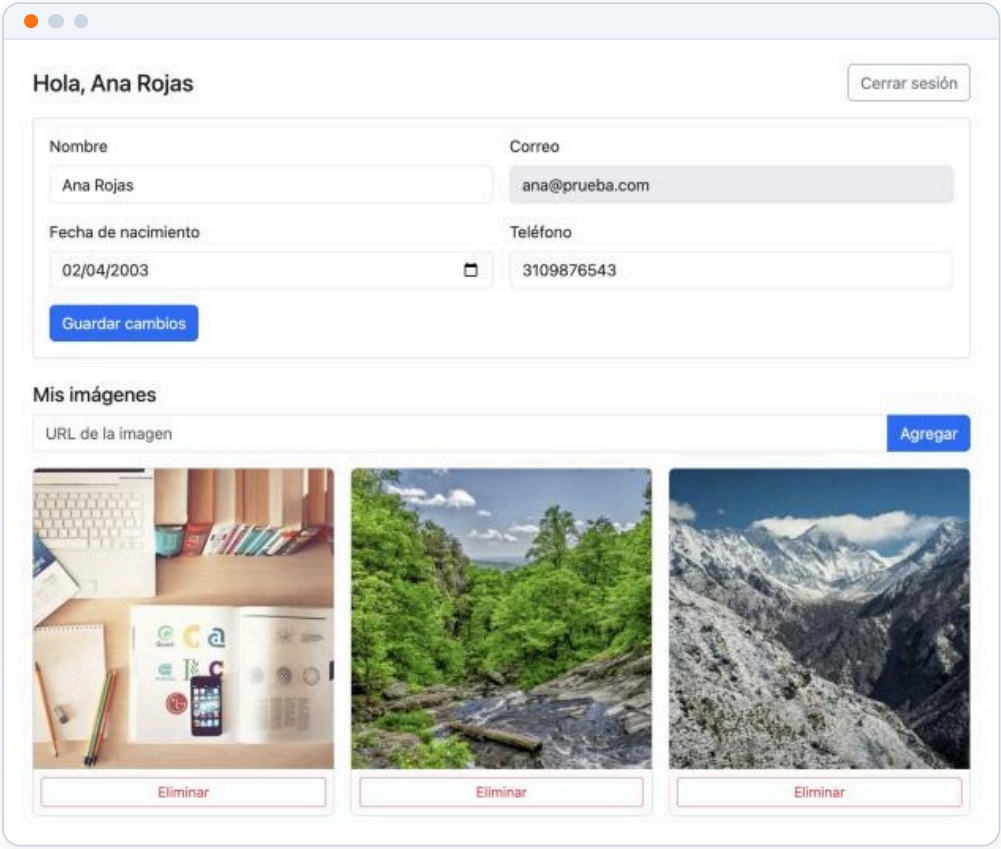
  return (
    <div className="container py-5">
      <div className="row justify-content-center">
        <div className="col-md-5">
          <div className="card shadow-sm">
            <div className="card-body p-4">
```

```
              <h2 className="h4 mb-3">Crear cuenta</h2>
              <form onSubmit={handleRegistro} className="d-grid gap-2">
                <input className="form-control" name="nombre"
                  placeholder="Nombre" value={form.nombre}
                  onChange={handleChange} required />
                <input className="form-control" name="correo" type="email"
                  placeholder="Correo" value={form.correo}
                  onChange={handleChange} required />
                <input className="form-control" name="password"
                  type="password" placeholder="Contraseña (mínimo 6)"
                  value={form.password} onChange={handleChange} required />
                <input className="form-control" name="fecha_nacimiento"
                  type="date" value={form.fecha_nacimiento}
                  onChange={handleChange} />
                <input className="form-control" name="telefono"
                  placeholder="Teléfono" value={form.telefono}
                  onChange={handleChange} />
                <button className="btn btn-primary">Registrarme</button>
              </form>
              {mensaje && <div className="alert alert-info mt-3">{mensaje}</div>}
              <button className="btn btn-link px-0 mt-2"
                onClick={() => navigate('/login')}>
                Ya tengo cuenta
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>
  )
}

export default Registro
```

Perfil

Carga los datos de quien inició sesión, deja editarlos y administrar sus imágenes. Fíjate que el `update` no envía el rol: aunque lo enviara, el trigger `proteger_rol` lo rechazaría. Si el usuario es admin, aparece el botón **Ir al dashboard**.



Perfil de Ana, una usuaria normal: ve solo sus 3 imágenes y no tiene el botón del dashboard.

src/componentes/perfil/index.jsx

```jsx
// src/componentes/perfil/index.jsx
import { useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { supabase } from '../../supabase'

function Perfil() {
  const [perfil, setPerfil] = useState(null)
  const [imagenes, setImagenes] = useState([])
  const [nuevaUrl, setNuevaUrl] = useState('')
  const navigate = useNavigate()

  useEffect(() => {
    async function cargar() {
      const { data: { user } } = await supabase.auth.getUser()
      const { data } = await supabase
        .from('usuario')
        .select('*')
        .eq('id', user.id)
        .single()
      setPerfil(data)
      cargarImagenes(user.id)
    }
```

```
    cargar()
  }, [])

  const cargarImagenes = async (usuarioId) => {
    const { data } = await supabase
      .from('multimedia')
      .select('*')
      .eq('usuario_id', usuarioId)
      .order('creado_en', { ascending: false })
    setImagenes(data ?? [])
  }

  const handleChange = (e) =>
    setPerfil({ ...perfil, [e.target.name]: e.target.value })

  const guardar = async () => {
    const { error } = await supabase
      .from('usuario')
      .update({
        nombre: perfil.nombre,
        fecha_nacimiento: perfil.fecha_nacimiento,
        telefono: perfil.telefono,
      })
      .eq('id', perfil.id)
    alert(error ? 'No se pudo guardar' : 'Datos actualizados')
  }

  const agregarImagen = async () => {
    if (!nuevaUrl.trim()) return
    const { error } = await supabase
      .from('multimedia')
      .insert({ url: nuevaUrl, usuario_id: perfil.id })
    if (!error) {
      setNuevaUrl('')
      cargarImagenes(perfil.id)
    }
  }

  const borrarImagen = async (id) => {
    const { error } = await supabase.from('multimedia').delete().eq('id', id)
    if (!error) setImagenes(imagenes.filter((img) => img.id !== id))
  }

  const cerrarSesion = async () => {
    await supabase.auth.signOut()
    navigate('/login')
  }

  if (!perfil) return <p className="p-4">Cargando...</p>

  return (
    <div className="container py-4">
      <div className="d-flex justify-content-between align-items-center mb-3">
        <h2 className="h4 mb-0">Hola, {perfil.nombre}</h2>
        <div className="d-flex gap-2">
          {perfil.rol === 'admin' && (
            <button className="btn btn-primary"
              onClick={() => navigate('/dashboard')}>Ir al dashboard</button>
          )}
          <button className="btn btn-outline-secondary"
            onClick={cerrarSesion}>Cerrar sesión</button>
        </div>
      </div>
```

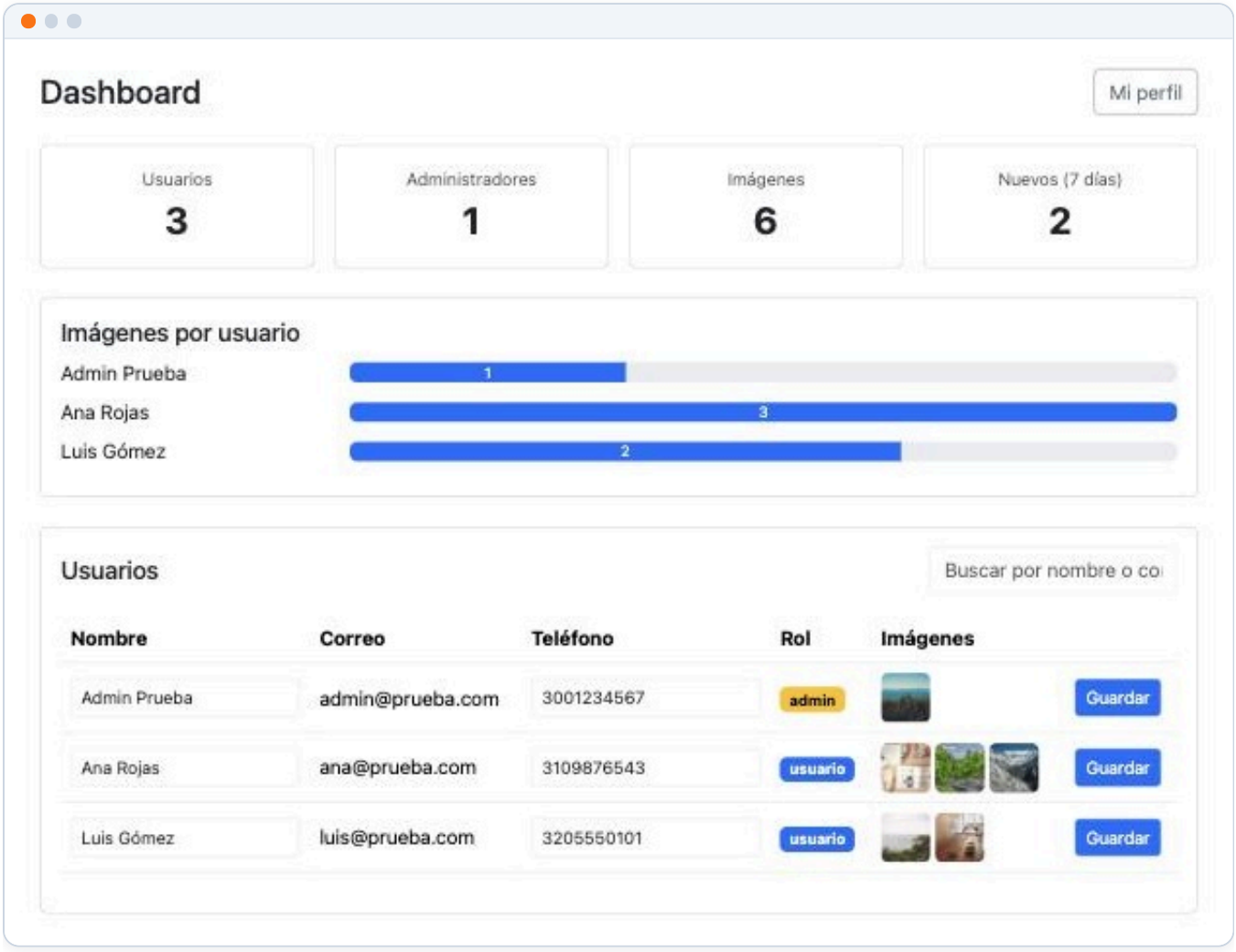
```
      <div className="card mb-4">
        <div className="card-body row g-3">
          <div className="col-md-6">
            <label className="form-label">Nombre</label>
            <input className="form-control" name="nombre"
              value={perfil.nombre} onChange={handleChange} />
          </div>
          <div className="col-md-6">
            <label className="form-label">Correo</label>
            <input className="form-control" value={perfil.correo} disabled />
          </div>
          <div className="col-md-6">
            <label className="form-label">Fecha de nacimiento</label>
            <input className="form-control" type="date"
              name="fecha_nacimiento" value={perfil.fecha_nacimiento ?? ''}
              onChange={handleChange} />
          </div>
          <div className="col-md-6">
            <label className="form-label">Teléfono</label>
            <input className="form-control" name="telefono"
              value={perfil.telefono ?? ''} onChange={handleChange} />
          </div>
          <div className="col-12">
            <button className="btn btn-primary" onClick={guardar}>
              Guardar cambios
            </button>
          </div>
        </div>
      </div>

      <h3 className="h5">Mis imágenes</h3>
      <div className="input-group mb-3">
        <input className="form-control" placeholder="URL de la imagen"
          value={nuevaUrl} onChange={(e) => setNuevaUrl(e.target.value)} />
        <button className="btn btn-primary" onClick={agregarImagen}>
          Agregar
        </button>
      </div>
      <div className="row row-cols-2 row-cols-md-3 g-3">
        {imagenes.map((img) => (
          <div className="col" key={img.id}>
            <div className="card h-100">
              <img src={img.url} className="card-img-top" alt="" />
              <div className="card-body p-2">
                <button className="btn btn-sm btn-outline-danger w-100"
                  onClick={() => borrarImagen(img.id)}>Eliminar</button>
              </div>
            </div>
          </div>
        ))}
      </div>
    </div>
  )
}

export default Perfil
```

9 El dashboard

Es el entregable principal. Tiene cuatro partes, numeradas en los comentarios del código.



Dashboard del admin, solo con Bootstrap. Indicadores arriba, gráfica de barras en el medio y tabla editable con buscador abajo.

1 **Solo entra el admin.** Consulta el rol de quien tiene la sesión; si no es `admin`, lo devuelve al perfil.

2 **Una sola consulta con join.** `select('..., multimedia(id, url)')` trae cada usuario con sus imágenes, porque existe la llave foránea. No hace falta pedir las dos tablas y cruzarlas a mano.

3 **Indicadores.** Se calculan en JavaScript con `filter`, `reduce` y `length` sobre los datos que ya llegaron.

4 **Editar y borrar.** Funcionan porque las políticas de RLS dejan al admin tocar las filas de todos.

Prueba que RLS funciona

Inicia sesión como `ana@prueba.com` y escribe `/dashboard` en la barra de direcciones: te devuelve al perfil. Luego, en `Perfil`, quita por un momento el `.eq('id', user.id)` y el `.single()` e imprime el resultado con `console.log`: aunque la consulta no tenga filtro, Ana solo recibe **su** fila. Eso es RLS trabajando en el servidor.

src/componentes/dashboard/index.jsx

```jsx
// src/componentes/dashboard/index.jsx
import { useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { supabase } from '../../supabase'

function Dashboard() {
  const [usuarios, setUsuarios] = useState([])
  const [busqueda, setBusqueda] = useState('')
  const [cargando, setCargando] = useState(true)
  const navigate = useNavigate()

  // 1. Solo entra el admin
  useEffect(() => {
    async function verificarYCargar() {
      const { data: { user } } = await supabase.auth.getUser()
      const { data: yo } = await supabase
        .from('usuario').select('rol').eq('id', user.id).single()

      if (yo?.rol !== 'admin') {
        navigate('/perfil')
        return
      }
      await cargarDatos()
      setCargando(false)
    }
    verificarYCargar()
  }, [])

  // 2. Usuarios con sus imágenes en UNA consulta (join por llave foránea)
  const cargarDatos = async () => {
    const { data, error } = await supabase
      .from('usuario')
      .select('id, nombre, correo, telefono, rol, creado_en, multimedia(id, url)')
      .order('creado_en', { ascending: false })
    if (!error) setUsuarios(data)
  }

  // 3. Indicadores calculados con los datos que ya tenemos
  const totalImagenes = usuarios.reduce((s, u) => s + u.multimedia.length, 0)
  const admins = usuarios.filter((u) => u.rol === 'admin').length
  const haceUnaSemana = Date.now() - 7 * 24 * 60 * 60 * 1000
  const nuevos = usuarios.filter(
    (u) => new Date(u.creado_en) > haceUnaSemana
  ).length
  const maxImagenes = Math.max(1, ...usuarios.map((u) => u.multimedia.length))

  const indicadores = [
    ['Usuarios', usuarios.length],
    ['Administradores', admins],
    ['Imágenes', totalImagenes],
    ['Nuevos (7 días)', nuevos],
```

```
  ]

  const filtrados = usuarios.filter((u) =>
    `${u.nombre} ${u.correo}`.toLowerCase().includes(busqueda.toLowerCase())
  )

  // 4. Editar y borrar (RLS lo permite porque somos admin)
  const cambiarCampo = (id, campo, valor) =>
    setUsuarios(usuarios.map((u) =>
      (u.id === id ? { ...u, [campo]: valor } : u)))

  const guardar = async (u) => {
    const { error } = await supabase
      .from('usuario')
      .update({ nombre: u.nombre, telefono: u.telefono })
      .eq('id', u.id)
    alert(error ? 'No se pudo guardar' : `Guardado: ${u.nombre}`)
  }

  const borrarImagen = async (imagenId) => {
    const { error } = await supabase
      .from('multimedia').delete().eq('id', imagenId)
    if (!error) cargarDatos()
  }

  if (cargando) return <p className="p-4">Cargando...</p>

  return (
    <div className="container py-4">
      <div className="d-flex justify-content-between align-items-center mb-4">
        <h1 className="h3 mb-0">Dashboard</h1>
        <button className="btn btn-outline-secondary"
          onClick={() => navigate('/perfil')}>Mi perfil</button>
      </div>

      <div className="row g-3 mb-4">
        {indicadores.map(([titulo, valor]) => (
          <div className="col-6 col-md-3" key={titulo}>
            <div className="card text-center h-100">
              <div className="card-body">
                <div className="text-muted small">{titulo}</div>
                <div className="fs-2 fw-bold">{valor}</div>
              </div>
            </div>
          </div>
        ))}
      </div>

      <div className="card mb-4">
        <div className="card-body">
          <h2 className="h5">Imágenes por usuario</h2>
          {usuarios.map((u) => (
            <div className="d-flex align-items-center gap-2 mb-2" key={u.id}>
              <span className="w-25 text-truncate">{u.nombre}</span>
              <div className="progress flex-grow-1">
                <div className="progress-bar"
                  style={{ width: `${(u.multimedia.length / maxImagenes) * 100}%` }}>
                  {u.multimedia.length}
                </div>
              </div>
            </div>
          ))}
        </div>
```

```
      </div>

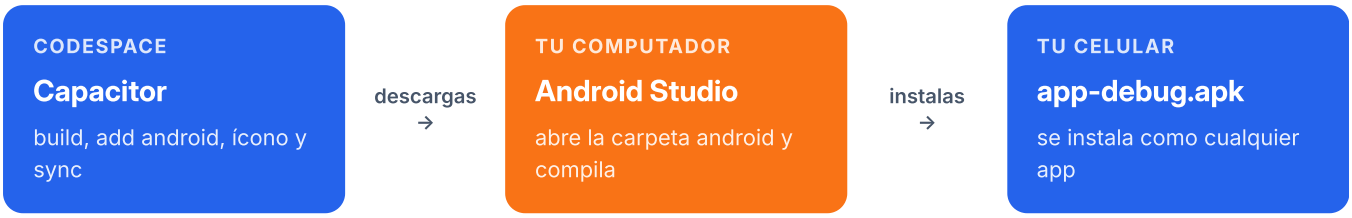
      <div className="card">
        <div className="card-body">
          <div className="d-flex justify-content-between align-items-center gap-3">
            <h2 className="h5 mb-0">Usuarios</h2>
            <input className="form-control w-auto"
              placeholder="Buscar por nombre o correo" value={busqueda}
              onChange={(e) => setBusqueda(e.target.value)} />
          </div>
          <div className="table-responsive">
            <table className="table table-hover align-middle mt-3">
              <thead>
                <tr>
                  <th>Nombre</th><th>Correo</th><th>Teléfono</th>
                  <th>Rol</th><th>Imágenes</th><th></th>
                </tr>
              </thead>
              <tbody>
                {filtrados.map((u) => (
                  <tr key={u.id}>
                    <td>
                      <input className="form-control form-control-sm"
                        value={u.nombre} onChange={(e) =>
                          cambiarCampo(u.id, 'nombre', e.target.value)} />
                    </td>
                    <td>{u.correo}</td>
                    <td>
                      <input className="form-control form-control-sm"
                        value={u.telefono ?? ''} onChange={(e) =>
                          cambiarCampo(u.id, 'telefono', e.target.value)} />
                    </td>
                    <td>
                      <span className={`badge ${u.rol === 'admin'
                        ? 'text-bg-warning' : 'text-bg-primary'}`}>
                        {u.rol}
                      </span>
                    </td>
                    <td>
                      {u.multimedia.map((img) => (
                        <img key={img.id} src={img.url} alt=""
                          width="40" height="40" title="Clic para borrar"
                          className="rounded object-fit-cover me-1"
                          onClick={() => borrarImagen(img.id)} />
                      ))}
                    </td>
                    <td>
                      <button className="btn btn-sm btn-primary"
                        onClick={() => guardar(u)}>Guardar</button>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </div>
      </div>
    </div>
  )
}

export default Dashboard
```

Parte del dashboard	Clases de Bootstrap que usa
Indicadores	`row g-3`, `col-6 col-md-3`, `card text-center`, `fs-2 fw-bold`
Gráfica de barras	`progress` y `progress-bar`; el ancho de cada barra es el único `style` del archivo
Tabla	`table-responsive`, `table table-hover align-middle`, `form-control-sm`
Rol	`badge text-bg-warning` para admin y `text-bg-primary` para usuario

10 Convertir en app Android

Capacitor toma la carpeta `dist` (el HTML, CSS y JS que genera `npm run build`) y la mete dentro de un proyecto Android. La app es tu misma página corriendo en un navegador interno, con su ícono y su pantalla de carga.



Todos los comandos de Capacitor corren en el Codespace: tu computador no necesita Node. Android Studio solo se usa al final, para compilar el APK. Si tu equipo no lo aguanta, el paso 9 tiene otra opción.

En el Codespace

1. Haz commit y push de lo que tengas y crea una rama nueva para la app (en la barra de abajo, clic en *main* → **Create new branch**). Capacitor crea muchos archivos y así no ensucias `main`.

2. Instala Capacitor y su generador de íconos:

```
Terminal
npm install @capacitor/core @capacitor/android
npm install -D @capacitor/cli @capacitor/assets
```

3. Inicializa Capacitor con el nombre de la app y su id. El id es único por app, al revés de un dominio: `com.tunombre.tuapp`, sin espacios ni tildes. `--web-dir dist` le dice dónde deja Vite la app compilada.

```
Terminal
npx cap init "Dashboard" "com.miproyecto.dashboard" --web-dir dist
```

Se crea este archivo en la raíz:

```
capacitor.config.json
{
  "appId": "com.miproyecto.dashboard",
  "appName": "Dashboard",
  "webDir": "dist"
}
```

4 Crea una carpeta `assets` en la raíz del proyecto (no dentro de `src`) y sube estas cinco imágenes PNG con estos nombres exactos (clic derecho sobre la carpeta → **Upload**):

```
assets/

assets/
├── icon-only.png          1024 × 1024
├── icon-foreground.png    1024 × 1024
├── icon-background.png    1024 × 1024
├── splash.png             2732 × 2732
└── splash-dark.png        2732 × 2732
```

- **icon-only:** el ícono completo, con fondo.
- **icon-foreground / background:** el dibujo sin fondo y el fondo solo. Android los combina y les da forma redonda o cuadrada.
- **splash:** la pantalla mientras la app carga. Deja el logo pequeño en el centro: los bordes se recortan según el celular.
- Para cambiar el tamaño sirve cualquier editor o un sitio como bulkresizephotos.com.

5 Compila la app web, crea el proyecto Android, genera el ícono y el splash, y copia la app dentro de Android. Corre los comandos uno por uno:

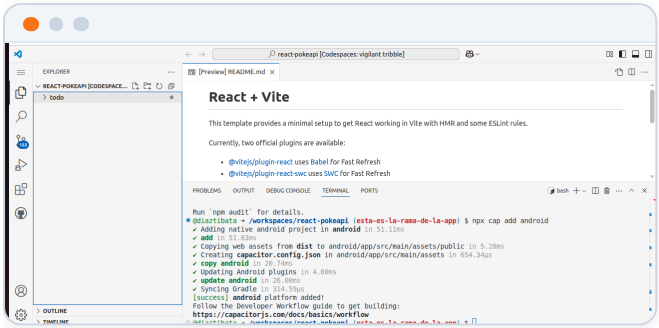
```
Terminal

npm run build
npx cap add android
npx capacitor-assets generate --android
npx cap sync android
```

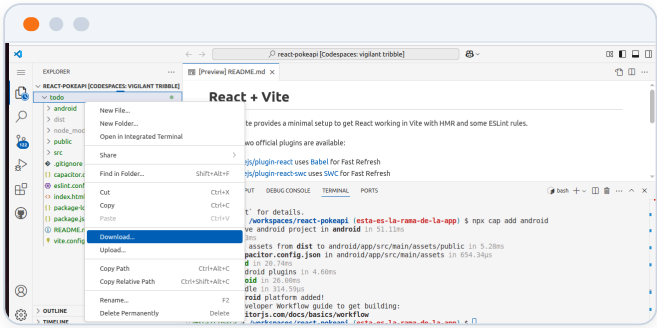
Comando	Qué hace
`npm run build`	Crea `dist` con tus claves de Supabase adentro. Necesita el `.env` lleno.
`npx cap add android`	Crea la carpeta `android`, un proyecto de Android Studio.
`npx capacitor-assets generate`	Lee la carpeta `assets` y crea el ícono y el splash en todos los tamaños.
`npx cap sync android`	Copia `dist` dentro de Android. Repítelo (junto con el build) cada vez que cambies la app.

Descargar y compilar en Android Studio

6 En el explorador del Codespace crea una carpeta nueva (por ejemplo `todo`), mueve ahí **todos** los archivos del proyecto y descárgala: clic derecho → **Download**. Tarda un buen rato porque incluye `node_modules`.



Todo dentro de una carpeta. Aquí se llama `todo` y queda sola en el explorador.



Clic derecho → Download. Abajo en la barra azul se ve el avance de la descarga.

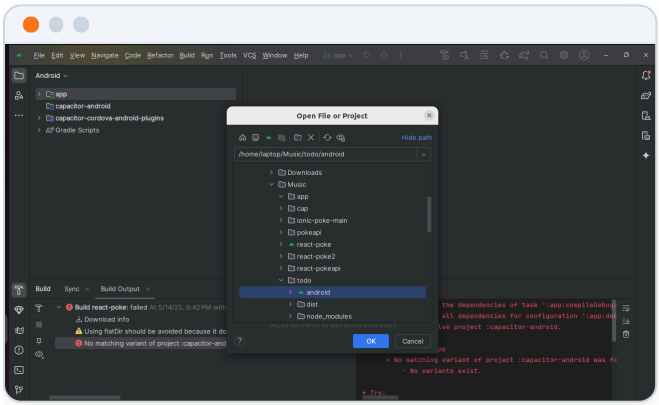
No esperes a que termine la descarga

En la barra azul de abajo del Codespace aparece el avance, por ejemplo *Downloading: 5841 of 7348 files*. Cuando vaya en unos **5.000 o 6.000 archivos** ya puedes abrir el proyecto en Android Studio y generar el APK: lo que falta son archivos de `node_modules` que el APK no usa.

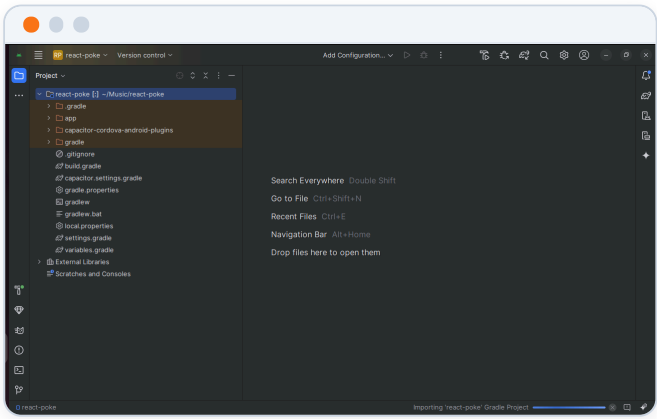
¿Por qué todo el proyecto y no solo la carpeta android?

El proyecto Android de Capacitor busca sus librerías en `../node_modules/@capacitor/android`. Si descargas solo `android`, Android Studio falla con errores como *No matching variant of project :capacitor-android*. Con la carpeta completa, esas rutas existen.

7 Descomprime la carpeta. En Android Studio ve a **File → Open** y abre la carpeta `android` que está **adentro** del proyecto descargado, no la carpeta del proyecto. Espera a que termine de cargar: la barra azul de abajo a la derecha desaparece y nada queda en naranja. Usa la última versión de Android Studio.

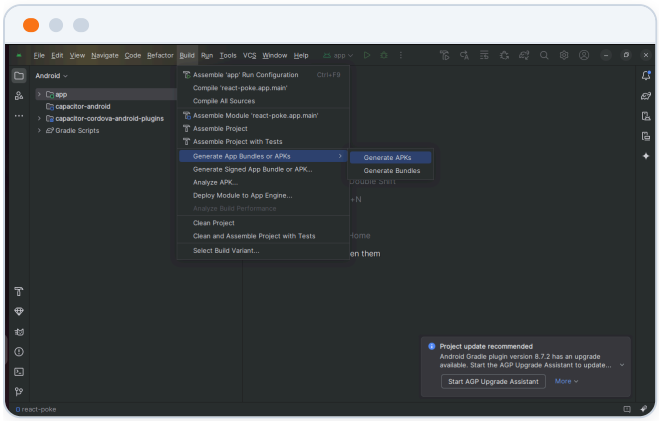


File → Open. Selecciona `todo/android`, no `todo`.

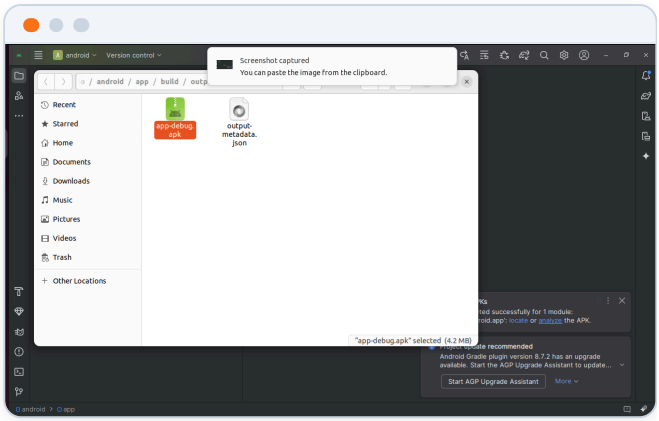


Cargando. Mientras las carpetas estén en naranja y la barra azul siga abajo a la derecha, espera.

8 En la barra de arriba: **Build → Generate App Bundles or APKs → Generate APKs**. Cuando termine, haz clic en **locate** en el aviso de abajo a la derecha. El archivo es `android/app/build/outputs/apk/debug/app-debug.apk`. Pásalo al celular y ábrelo; Android pedirá permiso para **instalar apps de origen desconocido**.



Build → Generate App Bundles or APKs → Generate APKs.



locate abre la carpeta con `app-debug.apk`.

Si tu computador no aguanta Android Studio

9 GitHub puede compilar el APK por ti con **GitHub Actions**, gratis en repos públicos. Crea el archivo `.github/workflows/apk.yml` (con el punto al inicio de `.github`) con este contenido:

.github/workflows/apk.yml

```yaml
# Compila el APK de Android en los servidores de GitHub.
# No necesitas Android Studio: ve a la pestaña Actions del repo.
name: Compilar APK

on:
  workflow_dispatch:
  push:
    branches: [main]

jobs:
  apk:
    runs-on: ubuntu-latest
    steps:
      - name: Descargar el código
        uses: actions/checkout@v4

      - name: Instalar Node
        uses: actions/setup-node@v4
        with:
          node-version: 22

      - name: Instalar Java (lo pide Android)
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 21

      - name: Instalar dependencias
        run: npm install

      - name: Compilar la app web (crea la carpeta dist)
        run: npm run build
```

```
      env:
        VITE_SUPABASE_URL: ${{ secrets.VITE_SUPABASE_URL }}
        VITE_SUPABASE_PUBLISHABLE_KEY: ${{ secrets.VITE_SUPABASE_PUBLISHABLE_KEY }}

    - name: Crear el proyecto Android
      run: npx cap add android

    - name: Generar ícono y splash desde la carpeta assets
      run: npx capacitor-assets generate --android

    - name: Copiar la app web dentro de Android
      run: npx cap sync android

    - name: Compilar el APK
      working-directory: android
      run: ./gradlew assembleDebug

    - name: Guardar el APK para descargarlo
      uses: actions/upload-artifact@v4
      with:
        name: app-debug
        path: android/app/build/outputs/apk/debug/app-debug.apk
```

10 El `.env` no se sube a GitHub, así que las claves van como **secretos**: en el repo, **Settings → Secrets and variables → Actions → New repository secret**. Crea `VITE_SUPABASE_URL` y `VITE_SUPABASE_PUBLISHABLE_KEY` con los valores de tu `.env`.

11 Haz push. En la pestaña **Actions** corre **Compilar APK** (unos 5 minutos). Cuando salga en verde, abre la ejecución y descarga **app-debug** en la sección *Artifacts*: es un .zip con el APK adentro.

El workflow hace lo mismo que los pasos 5 a 8

Instala dependencias, corre `npm run build`, `npx cap add android`, genera el ícono, hace `sync` y compila con `./gradlew assembleDebug`, que es lo mismo que el botón de Android Studio. Con esta opción agrega `android` al `.gitignore`: GitHub la vuelve a crear cada vez.

11 Errores comunes

Antes de preguntar, busca tu error aquí. Casi siempre es uno de estos.

Lo que ves	Por qué pasa	Cómo se arregla
`supabaseUrl is required`	Vite no leyó el `.env`.	Revisa que esté en la raíz, que las variables empiecen por `VITE_` y reinicia `npm run dev`.
La app se ve sin estilos	Falta cargar Bootstrap.	Revisa `npm install bootstrap` y el `import` en `main.jsx` (paso 3).
`Invalid API key`	La clave está incompleta o es de otro proyecto.	Cópiala otra vez desde **Connect**, sin espacios ni comillas.
La consulta devuelve `[]` sin error	RLS está filtrando: no hay sesión o la política no te deja ver esas filas.	Inicia sesión primero. Revisa las políticas en **Database → Policies**.
`new row violates row-level security policy`	Intentas insertar una fila que la política no permite.	En `multimedia`, `usuario_id` debe ser el id de quien tiene la sesión.
`Email not confirmed`	La cuenta se creó pero no se confirmó el correo.	Abre el correo de confirmación, o apaga **Confirm email** para practicar (paso 7).
`email rate limit exceeded`	El plan gratis envía pocos correos por hora.	Espera una hora, crea usuarios desde el panel o apaga **Confirm email**.
`Solo un admin puede cambiar el rol`	El trigger `proteger_rol` frenó un cambio de rol.	Así debe ser. Cambia el rol desde Table Editor.
`JSON object requested, multiple (or no) rows returned`	`.single()` no encontró la fila: el usuario existe en Auth pero no en `usuario`.	Pasa con cuentas creadas antes del trigger. Bórralas en Authentication y créalas de nuevo.
Recargo la página y me saca al login	`App.jsx` decide antes de saber si había sesión.	Usa el estado `cargando` como en la guía: no muestres rutas hasta que `getSession()` responda.
El APK abre en blanco	La app web falló al compilar o al cargar.	Corre `npm run build` y `npm run preview` en el Codespace y revisa la consola.
El APK abre pero no trae datos	Faltan los secretos o tienen otro nombre.	Revisa **Settings → Secrets** y vuelve a correr el workflow.
Subí el `.env` a GitHub	Faltaba en `.gitignore`.	Agrégalo, ejecuta `git rm --cached .env` y haz commit. Si subiste la clave secreta, genera una nueva en **Settings → API Keys**.

A Anexo: pedirle ayuda a Claude

Una IA ayuda mucho más cuando sabe en qué proyecto estás, con qué herramientas y qué no puede cambiar. Este anexo te deja listo ese contexto en el plan gratuito de Claude.

1. En claude.ai ve a **Customize → Connectors → GitHub Integration → Connect** e inicia sesión con tu GitHub. Es gratis y funciona con repos públicos.
2. En un chat nuevo haz clic en **+** (abajo a la izquierda) → **Add from GitHub** y pega `github.com/sdtibata/React-supabase`. Si tu propio repo es público, agrégalo también: así Claude compara el tuyo con el de referencia.
3. Copia este mensaje, completa lo que está entre corchetes y envíalo:

Mensaje para Claude

```
Soy estudiante y estoy haciendo la guía "Dashboard con React + Supabase".
El repositorio de referencia es github.com/sdtibata/React-supabase (te lo adjunto).

Mi contexto:
- Trabajo en GitHub Codespaces; mi computador no tiene Node.
- Stack: React + Vite, Supabase (Auth, PostgreSQL y RLS), Bootstrap 5 y Capacitor.
- Las claves van en .env con VITE_SUPABASE_URL y VITE_SUPABASE_PUBLISHABLE_KEY.
- Tablas: usuario (id = auth.users.id, nombre, correo, fecha_nacimiento,
  telefono, rol 'usuario' o 'admin', creado_en) y multimedia (id, url,
  usuario_id, creado_en). El perfil se crea con un trigger al registrarse.
- Estoy en la sección: [escribe el número y nombre de la sección].

Cómo quiero que me ayudes:
1. Antes de darme código, explícame en pocas líneas qué está pasando y por qué.
2. Compara con el repositorio de referencia y dime qué archivo y qué línea
   revisar. Cambia solo lo necesario; no reescribas todo el proyecto.
3. Usa solo clases de Bootstrap, sin CSS propio, y respeta los nombres de
   tablas, columnas y carpetas de la guía.
4. Nunca me pidas la clave secreta (sb_secret_) ni la pongas en el código.
5. Si te falta información, pregúntame el mensaje de error completo o el
   contenido del archivo antes de adivinar.

Mi problema:
[pega aquí el error completo de la terminal o de la consola del navegador
y el archivo donde ocurre]
```

Para que la respuesta sirva

Pega el error **completo**, no una foto ni "no funciona". Di en qué sección vas. Si te da código, pídele que te explique la línea que no entiendas antes de copiarla.

Lo que nunca le pegas

Tu clave secreta (`sb_secret_`), la contraseña de la base de datos ni contraseñas de usuarios. La URL y la clave publicable no son secretas, pero no hace falta pegarlas.